CE/CS/SE 3354 · Software Engineering · Fall 2026

Project Milestone P2

Analysis and High-Level Design

Game Review Aggregator

Team	Team 03, Section.010
Document version	1.0 (baseline for P2; grows from your P1 version 0.1)
Date	11/07/2026
Repository URL	AgentRachel/game-review-aggregator: Software Engineering project for aggregating and presenting trustworthy video game review information.
Git tag for this milestone	p2 (points at the commit that contains this exact PDF)
Submitted on Canvas by	Terrell Johnson

Team member	NetID	GitHub username
Terrell Johnson	tkj230000	ThatRell
Chayeol Jung	cxj240000	jcy1205
Mary Demas	dal919457	AgentRachel
Umar Shaikh	uas230000	MLGHackPolice
Rohith Rajan	rxr230118	taure0
Ethan Nguyen	ekn210003	ekn2022

0 Revision History, Contribution Record and Changes Since P1

All three tables below are graded. The contribution record carries the individual 20% of this milestone and must agree with your repository history.

Version	Date	Author	Summary of change
0.1	10/18/2026	Terrell Johnson, Mary Demas, Umar Shaikh, Rohith Rajan, Ethan Nguyen, Chayeol Jung	P1 baseline
1.0	11/8/2026	Terrell Johnson, Mary Demas, Umar Shaikh, Rohith Rajan, Ethan Nguyen, Chayeol Jung	P2 baseline: analysis and high-level design added

0.1 Individual contribution record (P2)

Team member	Sections and artifacts this person produced for P2	Evidence in the repository (commit hashes, PR numbers)	Reviews given (PR numbers)
Terrell Johnson	Section 5 First half. Section 8	Issues: 38, 39, 50	PR #54, #59, #33
Chayeol Jung	Section 1-4, Section 5 Second half	Issues: 36,37,40, 41, 55	
Mary Demas	Use Cases scenario walkthroughs for section 4	Commit hash: dfc9a8b	
Umar Shaikh	Section 6	PR: #59, Commit hash: 19ad558, Issue 60	
Rohith Rajan	Section 7: remaining parts	Commit hash: 719d1cb	
Ethan Nguyen	11, Section 5 first half, Parts of section 7, Section 10.3	Issue 40, 56	

0.2 Changes since the P1 baseline

Change ID	What changed (IDs affected)	Why (P1 feedback, new finding, spike result)	Knock-on effect	Agreed on
CH-01	Section 3.4	P1 feedback, needed to add number to use cases	Traceability commentary	11/8/2026
CH-02	Section 1.2, FR-07	P1 feedback, reworded scoped feature “Allow users to compare the available ratings from different sources for a given game” to be more concise	FR-07 now reflects this in-scope feature	11/8/2026
CH-03	Section 3.1, FR-10	P1 feedback, added missing requirement FR-10 for a scoped feature	Traceability commentary	11/8/2026

1 to 3 Introduction, Overall Description and Requirements (carried forward)

1 Introduction

1.1 Purpose of this document and of the system

The purpose of this document is to online the requirements, goals, and the functions of the Game Review Aggregator system. It is intended for the development team, the stakeholders, and anyone to review or evaluate the system. This document will serve as a reference throughout the development process to ensure it meets the intended requirements.

The Game Review Aggregator is a website that is designed to help gamers research, discover, and compare video games by bringing reviews from several different sources into one place. Currently, games must visit several other websites to compare ratings and opinions, which is time-consuming and confusing as often reviews conflict with each other. This system aims to solve this problem by allowing users to search for games, filter results based on their preferences, and view ratings and detailed reviews from available sources on one location.

Unchanged since v0.1

1.2 Scope

In scope — we will build this	Out of scope — we will not, and why
A website where users can search for video games and view their information, and publish their own reviews	A game purchasing or payment system, since the website is intended for research rather than selling games.
Display available game reviews, ratings, and their original sources	Integration with every review platform, because API restrictions and development time limitations
Allow users to filter games by selected characteristics, such as genre and platform, when the data is available	Advanced personalized recommendation algorithms, because they require additional development and testing
Allow users to compare the available ratings from different sources for a given game	Automatic detection of fake or biased reviews, because accurately identifying them would require more complex methods
Display review details, including written explanations and publication dates when available	Steam account login and friend-based review prioritization, because their

In scope — we will build this	Out of scope — we will not, and why
	necessity and technical feasibility have not been confirmed

These features were excluded to make the project manageable within the course timeline and allow the development team to focus on the core functionality of the website. More advanced features could be considered in future versions once the basic system is out.

1.3 Definitions, acronyms and abbreviations

Term or abbreviation	Meaning as used in this project
Game Review Aggregator	A website that collects and displays video game reviews and ratings from multiple sources in one location
API (Application Programming Interface)	A way for the website to retrieve game information and reviews from external platforms such as Steam
Aggregate Rating	A combined rating calculated using reviews or scores from multiple sources
Stakeholder	A person or organization that uses, develops, or is affected by the system, such as gamers, developers, or Steam
Review Source	The original platform or website where a game review or rating was published
Filtering	Narrowing game search results based on characteristics such as genre or platform
Steam	A digital gaming platform that may provide game reviews and information through its supported API
UI (User interface)	The visual parts of the website that users interact with, including search bars, filters, and review pages

Unchanged since v0.1

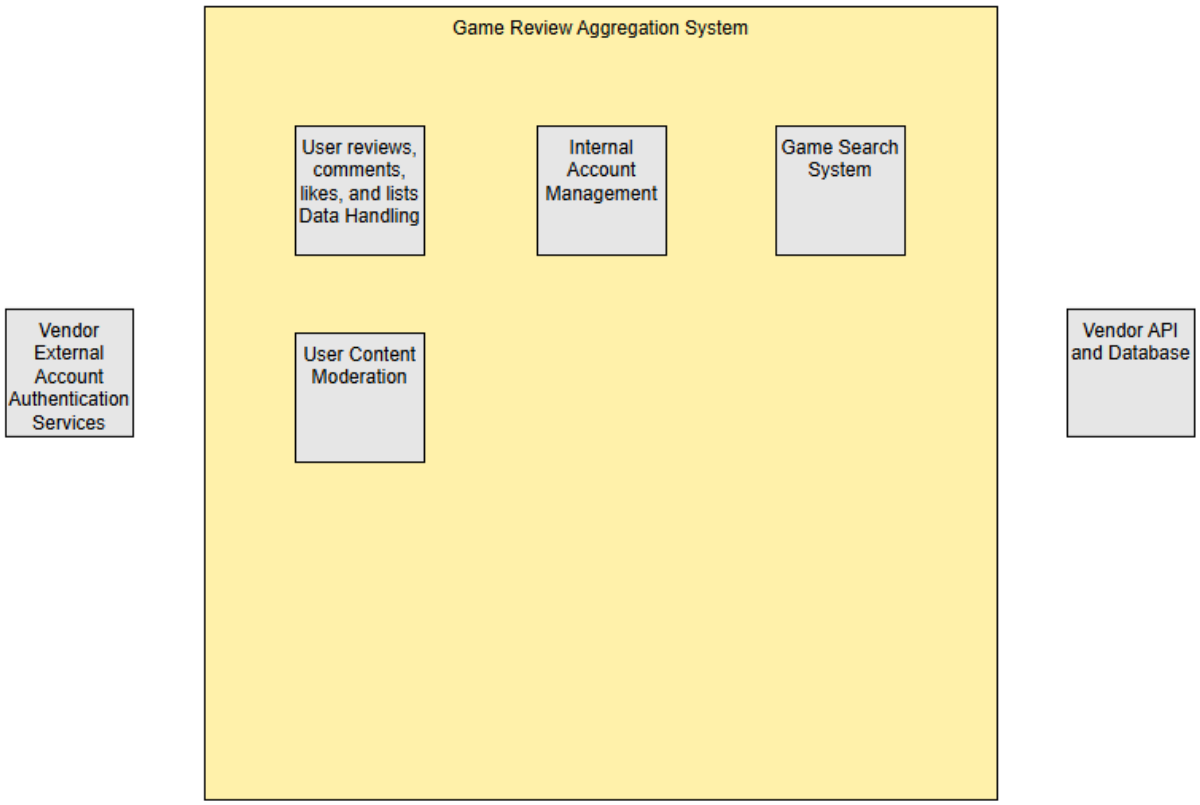
1.4 References

[R1] Valve Corporation, *Steamworks Web API Overview*. Steamworks Web API Documentation
[R2] Valve Corporation, *Steam User Reviews API*. Steam User Reviews API Documentation
[R3] Interview transcript in repository. docs\interviews\Interview_*1*_Gamer.pdf
[R3] Interview notes in repository. docs\interviews\Interview_2_Gamer.pdf

Unchanged since v0.1

2 Overall Description

2.1 Product perspective and context



The Game Review Aggregation system is only responsible for the handling of internal data, mainly the data tied to individual user accounts such as reviews, likes, and lists, as well as the account information. If users want to link external accounts from vendors like Steam or Roblox, that functionality wouldn't fall under our system responsibility since that would require external authentication services. Furthermore, game metadata supplied by databases and APIs from vendors such as Steam would allow us to provide the information we need for our game search system. However, this ultimately means that we are dependent and not responsible for the data we are supplied with. Lastly, account management and user content moderation are within the bounds of our system responsibility since these systems operate strictly on our data and would rely on no third-party providers.

Unchanged since v0.1

2.2 Stakeholders and how you reached them

Stakeholder	Role in relation to the system	What they need from it	How you learned this
Gamers researching games (represented by interview participants)	Primary users who use the website to research games before deciding what to play or purchase.	Reliable reviews, useful ratings, information about game difficulty and quality, and help deciding whether a game suits their preferences.	Interviews with two gamers (R1 and R2). Their gaming experiences and preferences informed the initial requirements.
Casual and dedicated game reviewers	Users who contribute written reviews and ratings.	An easy way to submit reviews and ratings that other gamers can read.	Initially assumed based on the proposed user-review functionality; needs confirmation through additional elicitation.
Website moderators	Responsible for reviewing reported or inappropriate content, if moderation is included.	Tools to identify and remove reviews that violate the website's rules.	Initially assumed based on the possibility of user-submitted reviews; moderator needs have not yet been directly elicited.
External game and review platforms (e.g., Steam)	Potential providers of game information or existing reviews if external data is incorporated.	Appropriate use of their information and compliance with applicable access restrictions and terms.	Identified during project planning. Specific platform requirements must be researched if external data is used.
Website administrators and development team	Responsible for maintaining the website and its data.	A manageable system that can be maintained, updated, and operated within the team's available resources.	Assumed based on the team's responsibility for developing and maintaining the project.

Unchanged since v0.1

2.3 User characteristics

Gamers researching games: These users visit the website to learn about games before deciding whether to play or purchase them. They may have different levels of gaming experience and may not be familiar with every game's mechanics, difficulty, or genre. They need clear game information, understandable ratings, and reviews that help them make informed decisions.

Game reviewers: These users submit written reviews and ratings based on their experiences with games. They may have different levels of writing experience and familiarity with review websites. The system should provide a straightforward review submission process with clear instructions and understandable rating options.

Website moderators: These users are responsible for managing user-submitted content and ensuring reviews follow the website's rules. They may have varying levels of technical experience, so moderation functions should be clearly labeled and easy to use.

Unchanged since v0.1

2.4 Constraints

We must comply with the deliverable requirements, submission format, and deadlines specified for SE 3354. We must maintain the project's source code and development history in the team's designated version-control repository. We must comply with the applicable terms of service and technical requirements of any external platform from which we retrieve data. If we use the Steam Web API (R1), we must comply with its applicable access requirements and restrictions. We must not depend on access to private or restricted third-party data unless we have the necessary authorization. We must comply with applicable data usage and attribution requirements for external platforms. We must protect user privacy and comply with applicable privacy requirements if we collect personal information. We must use review sources that provide an accessible and permitted method of obtaining their data. We must use hosting and database services available within our team's budget and resources.

Unchanged since v0.1

2.5 Assumptions and dependencies

#	Assumption or dependency	What breaks if it does not hold
A1	Availability of game metadata from official vendors (Dependency)	Wouldn’t be able to use official vendor APIs if they don’t provide public metadata. Would have to use third party databases
A2	Vendors supporting account linking (Dependency)	Wouldn’t be able to support external account linking if not supported
A3	Social actions for users, such as creating lists and liking lists from other users (Assumption)	If list features aren’t used for social or game discoverability purposes, then they’d be added for no reason. While interviews haven’t explicitly shown a

#	Assumption or dependency	What breaks if it does not hold
		need for these features, popular review websites such as Letterboxd or MyAnimeList include these features, so we are assuming they'd be useful for our system as well.

Unchanged since v0.1

2.6 Feasibility summary

Feasibility gate	Verdict	Evidence behind the verdict
Technical	Feasible	Steam documents multiple endpoints such as "IUserReviewsService.GetAppReviews" that would allow us to collect reviews using official steam API. Furthermore, multiple independent third-party database sources (SteamDB, itchDB, and Rolimon's) that would allow us to gather game metadata for search functionality. Lastly, comparable systems such as Letterboxd are already in production, which shows that our system is technically feasible
Operational	Feasible	Our system would require little to no operation or maintenance after deployment, due to the website and user workflow being small, straightforward, and simple. Furthermore, interviews (R1 and R2) show that there's a need for this type of system, which suggests that it will be used after deployment
Schedule	A narrow initial version is feasible	Given a little less than two months, it would be more feasible if the team prioritized one supported data source, core search/display functionality, and a simple review/rating system.
Economic / resource	Feasible	The cost of hosting a website is around $2-15/month which is within budget. Database services such as Supabase and MongoDB have free tiers which would satisfy the needs for our simple website.

Unchanged since v0.1

3 Specific Requirements

3.1 Functional requirements

ID	Requirement — one “shall” sentence	Priority	Source	Acceptance criterion
FR-01	The system shall allow users to search for games by title.	MUST	Gamer interviews	A tester searches for an existing game and sees matching results.
FR-02	The system shall display an individual game's information, including its title and available review information.	MUST	Gamer interviews	A tester selects a game and sees its title and available review information.
FR-03	The system shall display game reviews submitted by users.	MUST	Gamer interviews	A tester opens a game page and can read its submitted reviews.
FR-04	The system shall allow registered users to submit a review for a game.	MUST	Gamer interviews	A registered tester submits a review, and it appears on the appropriate game page.
FR-05	The system shall allow users to assign a rating to a game when submitting a review.	MUST	Gamer interviews	A tester submits a review with a rating, and the rating is displayed correctly.
FR-06	The system shall display an aggregate rating calculated from available game ratings.	SHOULD	Project goal	A tester verifies that the displayed aggregate matches the ratings used in the calculation.
FR-07	The system shall allow users to sort available reviews by a supported criterion (by external source, most liked, etc.)	COULD	Feature	A tester selects a sorting option and sees reviews in the selected order.
FR-08	The system shall allow designated moderators to remove reviews that violate the review rules.	SHOULD	Moderator stakeholder	A moderator removes a review, and it no longer appears among published reviews.

ID	Requirement — one “shall” sentence	Priority	Source	Acceptance criterion
FR-09	The system shall allow users to report and flag content	SHOULD	Moderator stakeholder	A tester verifies that there are options to report user content such as reviews, comments, and lists. After reporting, flagged content should be visible for moderators
FR-10	The system shall allow users to filter games through selected characteristics such as rating and categories such as genre and platform.	SHOULD	Feature	A tester verifies that the resulting games from a search align with the filters applied during the search

3.2 Quality requirements

ID	Requirement — threshold, condition, and measurement	Priority	Source	How it will be verified
QR-01	Under normal test conditions with up to 20 simultaneous users, at least 95% of game searches shall return results within 3 seconds, measured through 100 test searches.	SHOULD	Proposed performance target	Run 100 searches and record their response times.
QR-02	The system shall display its main pages correctly at viewport widths of 360 pixels and 1366 pixels, verified by testing each page at both widths.	MUST	Proposed usability requirement	Open each main page at both viewport widths and check for clipped or overlapping content.
QR-03	The system shall prevent users who are not authorized moderators from accessing moderator-only actions in 100% of the tested unauthorized-access attempts.	MUST	Proposed security requirement	Attempt moderator actions using an ordinary user account and an unauthenticated session.
QR-04	For a test set of 20 valid reviews, the system shall display all 20 reviews correctly after they are submitted and the relevant page is reloaded.	MUST	Proposed reliability requirement	Submit 20 reviews, reload the relevant page, and confirm that all reviews remain available.

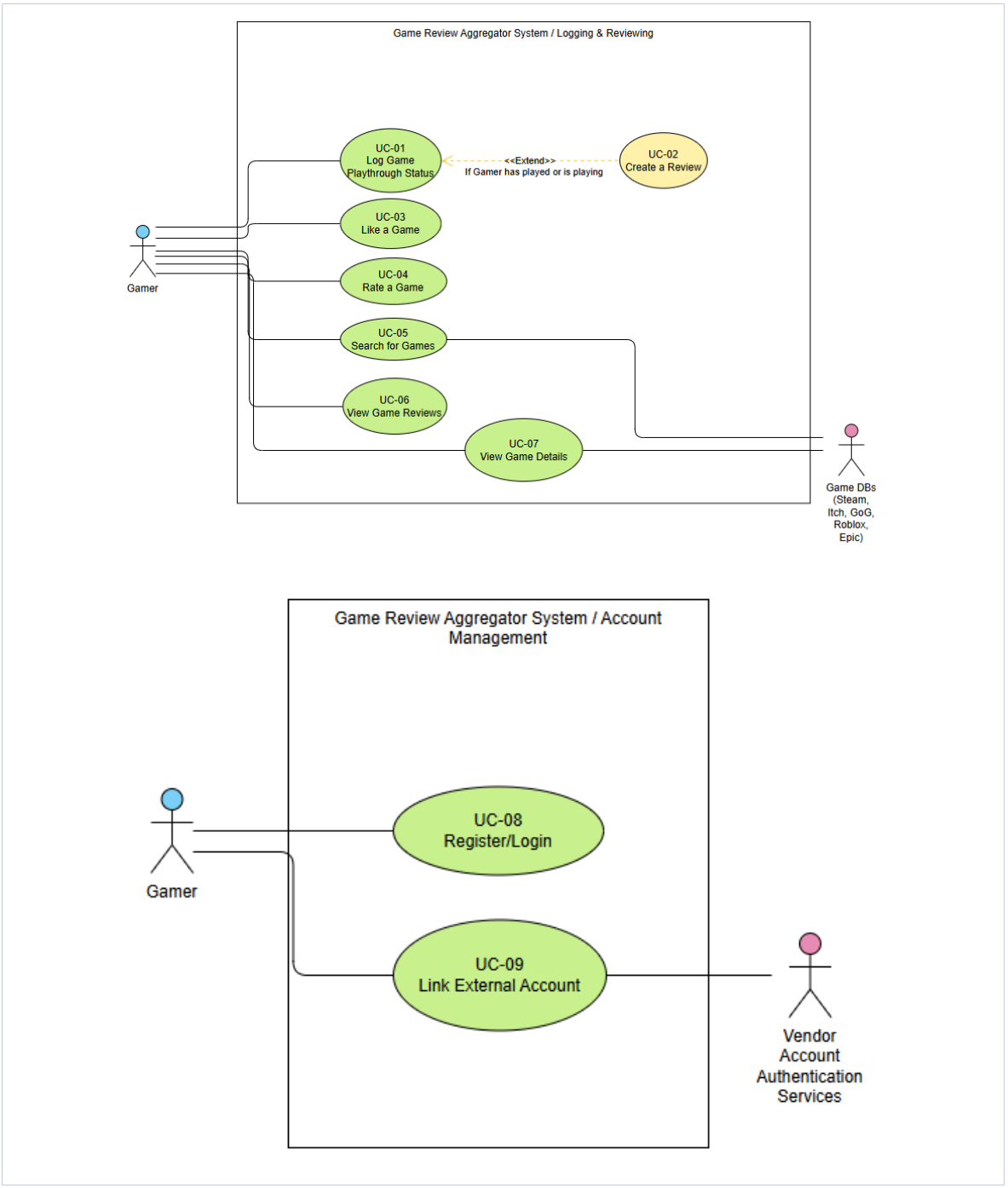
Unchanged since v0.1

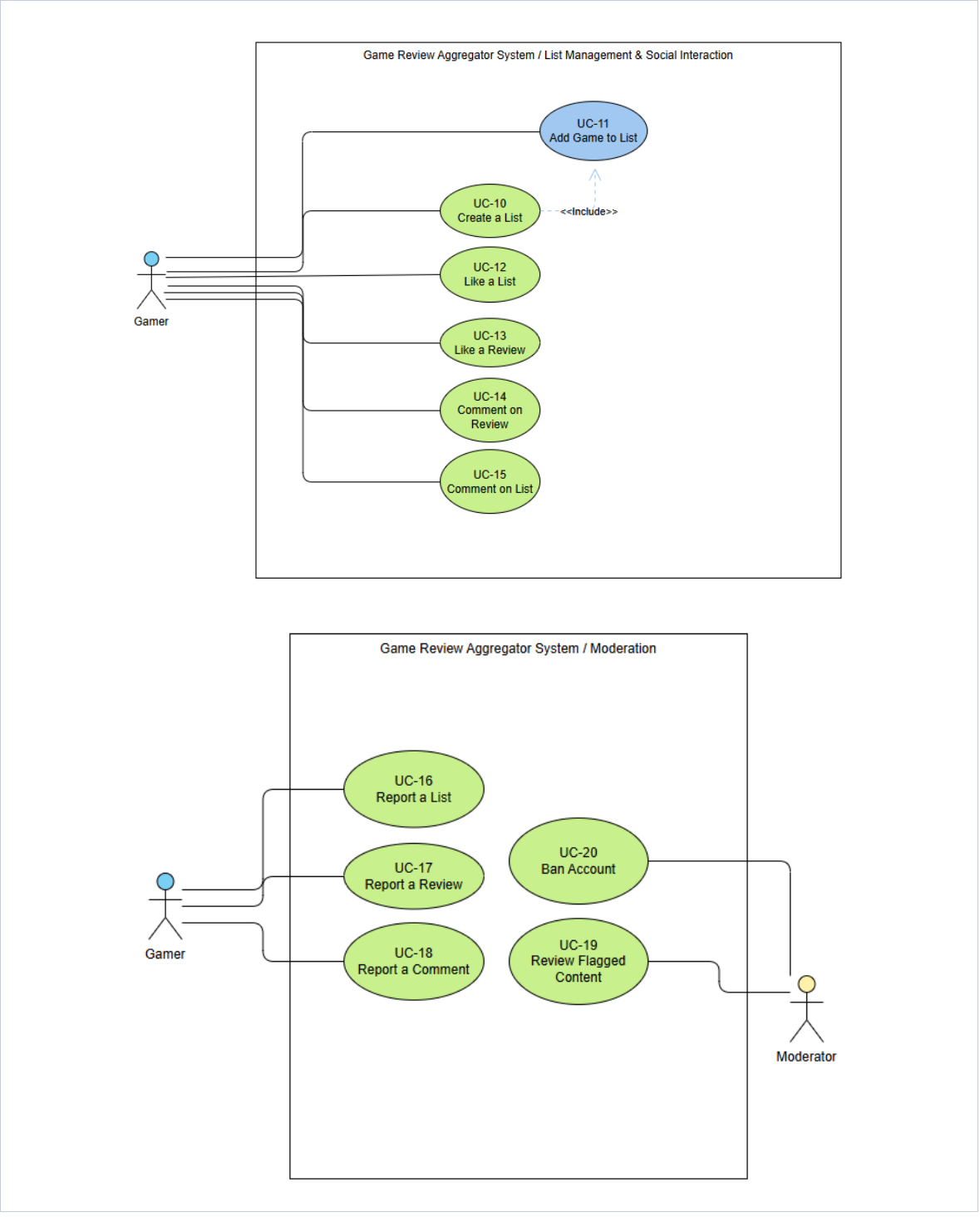
3.3 Constraint requirements

ID	Constraint — one sentence	Source	Why it cannot be negotiated
CON-01	The system shall comply with the applicable terms and access restrictions of any external platform from which it retrieves game or review data.	Official documentation and terms of each platform used, such as Steamworks documentation [R1]	The team cannot disregard the restrictions imposed by external data providers.
CON-02	The system shall not access private or restricted third-party game or review data without the authorization required by the data provider.	Official documentation and access policies of each platform used [R1]	The team cannot assume permission to access restricted data.

Unchanged since v0.1

3.4 Initial use cases





3.5 Requirements to use case traceability

9.1 Extended traceability matrix

4 Use-Case Analysis

Chapter 7, first half. Here you take the use cases from section 3.4 and write the most important of them out turn by turn, still with the technology switched off.

4.1 Use cases selected for analysis

Use case	Name	Why it was selected	Non-trivial steps
UC-06	View Game Reviews	It's the main use case of our system (carries most of the product).	System displays a game page, user looks through reviews, user gets review information
UC-05	Search for Games	Necessary use case for finding the games you want to see reviews for. Use case that includes FR-01, FR-10, and QR-01	User applies filters (optional), enters search term, system displays search results, user browses and picks a game, system opens game page
UC-02	Create a Review	It's another important use case that allows users to rate and create reviews for games (an important aspect of our product)	System displays the home page, user logs playthrough status as "Played" or "Playing", system allows user to create a review, user writes a review, system displays a message "review created, user acknowledges the message

4.2 Expanded use cases

Copy the block below once for each use case in 4.1.

UC-06 View Game Reviews

Primary actor	Gamer
Supporting actors and external systems	N/A
TUCBW (this use case begins when)	When a gamer is on a game page after searching and selecting a game
TUCEW (this use case ends when)	The gamer gets the desired review information and exists the game page
Preconditions	Gamer is on a game page
Postconditions	Gamer has seen the reviews and gotten the information they wanted
Traces to requirements	FR-02, FR-03, FR-07

Actor input / action	System response
	(0) System displays a game page
(1) Gamer navigates to the review portion of the page	(2) System displays a list of reviews
(3) Gamer sorts the reviews by recency	(4) System displays the sorted review list
(5) Gamer reads the reviews, and that is the acknowledgement	

Alternate flow	At step	Condition	What happens	Ends or rejoins at
A1	(1)	No reviews for the game	System shows “no reviews listed”	Gamer reads the message and that is acknowledgement
A2	(3)	Gamer doesn’t sort reviews	System shows a list defaulted to showing the most popular reviews	Rejoins at (4)

UC-05 Search for Games

Primary actor	Gamer
Supporting actors and external systems	Game DBs from Vendors
TUCBW (this use case begins when)	Gamer enters a search term in the search box
TUCEW (this use case ends when)	Gamer selects a game after searching and browsing the results
Preconditions	Gamer is on the home page, where they can use a search bar
Postconditions	Gamer selects a game they searched for and sees the game page information
Traces to requirements	FR-01, FR-10, QR-01

Actor input / action	System response
	(0) System displays a home screen
(1) Gamer enters a search term	(2) System displays a list of results corresponding to the search term

Actor input / action	System response
(3) Gamer applies filters	(4) System displays a list of results corresponding to the search term and filters applied
(5) Gamer selects a game	(6) System takes gamer to the game's page
(7) Gamer is on the game's page, this is acknowledgement and the use case ends	

Alternate flow	At step	Condition	What happens	Ends or rejoins at
A1	(3)	Gamer doesn't apply filters	System remains displaying all possible matching results	Rejoins at (5)
A2	(2)	There are no games that correspond to the search term	System displays "no matching results"	Gamer reads the message, that is acknowledgement and the use case ends

UC-02 Create a Review

Primary actor	Gamer
Supporting actors and external systems	N/A
TUCBW (this use case begins when)	Gamer logs a playthrough as "Played" or "Playing"
TUCEW (this use case ends when)	System shows that review is created
Preconditions	The gamer must log the game playthrough status as either "Played" or "Playing"
Postconditions	There's a new review created by the gamer
Traces to requirements	FR-04

Actor input / action	System response
	(0) System displays a home screen
(1) Gamer logs a playthrough as "Played"	(2) System displays a screen to write a review
(3) Gamer writes a review	(4) System displays a "review created"
(5) User reads the system message, which is the acknowledgement	

Alternate flow	At step	Condition	What happens	Ends or rejoins at
A1	(1)	The game that the gamer wants to log isn't in the system	System doesn't allow user to create a review for that game, displays a message "Game not found"	Gamer reads the message and the use case ends
A2	(1)	The gamer logs the game as "not played"	System doesn't allow user to create a review for that game, displays a message "Can't create review"	Gamer reads the message and the use case ends

4.3 Self-review log

Use case	Check from the Lesson 13 review checklist	Defect found	How it was fixed
UC-06	1, 2, 3, 4, 5, 6, 7, 8, 9, 10. All checked	n/a	n/a
UC-05	1, 2, 3, 4, 5, 6, 7, 8, 9. All checked	10. Named search box on the top of the screen	Had the system just display a generic home/idle page
UC-02	1, 2, 3, 4, 5, 6, 7, 8, 9. All checked	10. Initially had the gamer press a "Log" button to log a playthrough	Removed it and just had the gamer do the action "logs a playthrough"

5 Analysis Models

Chapter 7, second half. What has to exist inside the system for the use cases in section 4 to happen, and how those parts talk to each other.

5.1 Domain model

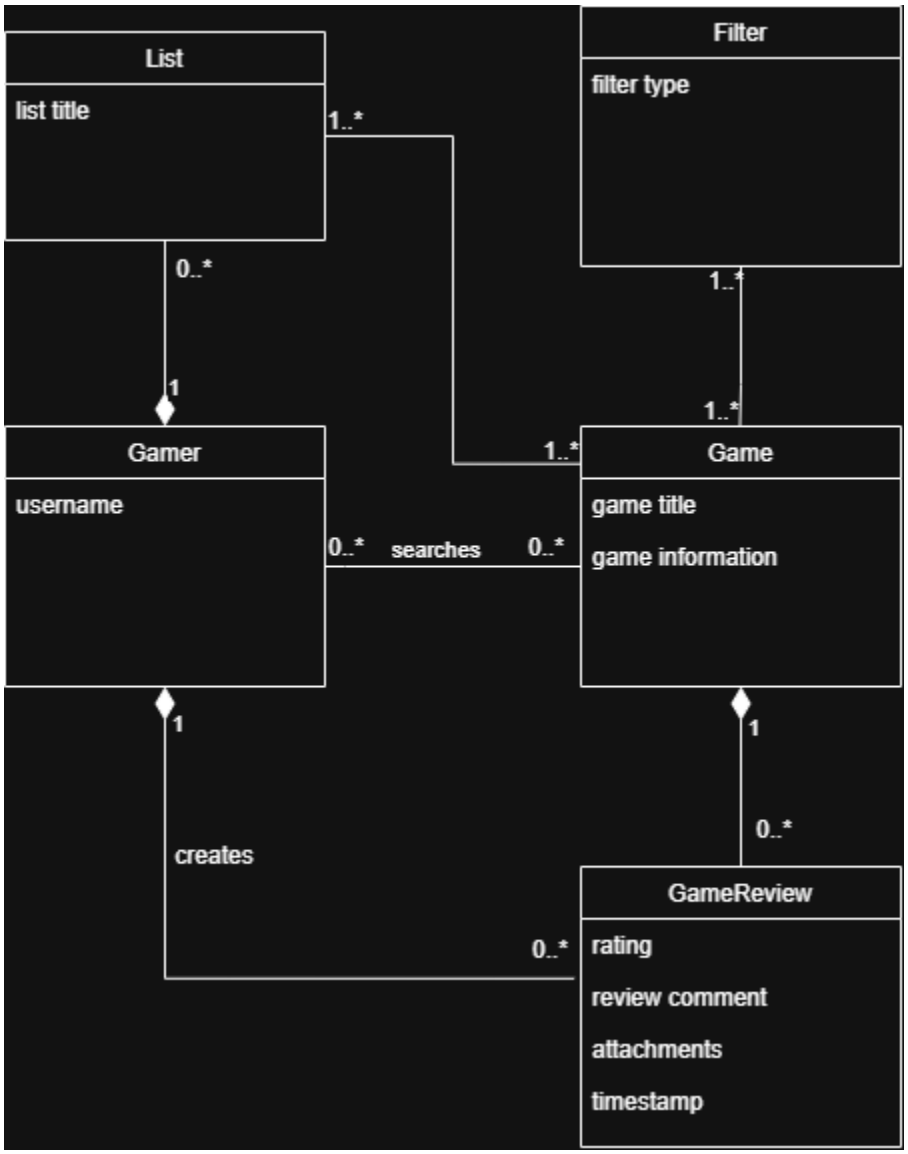


Figure 5.1 Domain model (conceptual classes, attributes, associations with multiplicities)

Class	What it represents (one line)	Key attributes	Associations (with multiplicity)
Gamer	User of system	username	Many users can search for many games. One user can create many reviews. One user can create many lists.

Class	What it represents (one line)	Key attributes	Associations (with multiplicity)
Game	A game that a user can look up	Game title, game information	Many games can be searched by many users. One game contains many game reviews (composition). A game can be connected to many lists.
GameReview	The reviews of a game listing	Rating, review	Game reviews can only belong to one game, and game reviews can only belong to one user (composition, the part dies with the whole)
Filter	Categories that a game belongs to	filter type	A filter can contain 1 or more games, and a game can belong to one or more filters
List	A list of games that a user can create and share	list name	One list can contain many games. A list can only belong to one user.

5.2 Object interaction models

Copy the block below for at least two of the use cases in 4.1; preferably including your riskiest.

5.2.x UC-xx [Use case name]

(a) Non-trivial steps. List the steps from 4.2 that need objects inside the system and say in one line why each of the others is trivial.

- **Step 2 (Non-trivial):** Requires system objects to parse search criteria, apply active genre and platform filters, query the catalog domain records, and rank matching candidates.
- **Step 3 (Non-trivial):** Requires the system to construct the structured search result set, calculate relative match scores, and format game summary entries for display.
- *(Step 0 is the passive pre-state; Step 1 is the physical entry of search criteria by the actor; Step 4 is the actor selecting a title from the returned list).*

(b) Scenario table.

Step				Subject (who acts)	Subject action (what it does)	Other data / objects (what is used)	Object acted upon (who receives)
2.1	Gamer	submits search keywords and filter criteria	SearchQuery parameters	SearchController			
2.2	SearchController	creates query evaluation instance	text, genreFilter, platformFilter	SearchQuery			
2.3	SearchController	queries catalog records matching title criteria	SearchQuery.text	Game			
2.4	SearchController	filters matching titles by selected platform	SearchQuery.platformFilter	Platform			

Step	Subject (who acts)	Subject action (what it does)	Other data / objects (what is used)	Object acted upon (who receives)
2.5	SearchController	filters matching titles by selected genre	SearchQuery.genreFilter	Genre
3.1	SearchController	creates ranked search result list	matching Game instances	SearchResult
3.2	SearchResult	retrieves summary details and aggregate score	title, aggregateScore	Game
3.3	SearchController	presents ranked matching game list	collection of SearchResult items	Gamer

(c) Analysis sequence diagram.

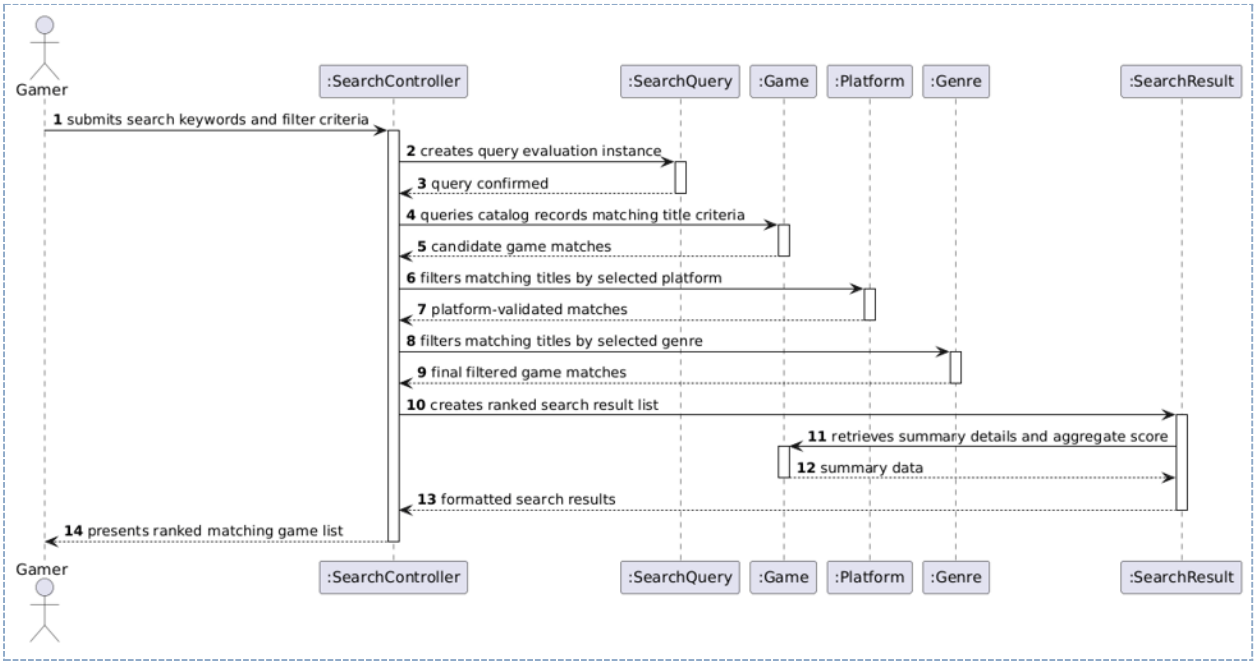


Figure 5.2.x Analysis sequence diagram for UC-xx; message numbers match the scenario table

5.3 Consistency cross-check

Check	Result (pass, or what failed)	Fixed in
K1 Every item in an actor column (4.2) is an attribute in the domain model (5.1)	Fail: UC-05 refers to a search term/search criteria, but the domain model does not clearly contain a corresponding attribute such as search text/query.	Add the needed concept/attribute to 5.1, or revise the wording in 4.2 so it maps cleanly to an existing domain concept.
K2 Every object on a sequence diagram is a domain class or a declared interaction object	Fail: The sequence diagram uses Platform and Genre, but those are not classes in the current domain model and are not clearly declared interaction objects.	Replace them with the existing Filter concept, or add/declare them properly in 5.1/5.2.
K3 Every message on a sequence diagram corresponds to exactly one scenario-table row	Pass: The numbered messages 2.1–2.5 and 3.1–3.3 correspond to the scenario-table rows.	N/A
K4 Every non-trivial step in 5.2 (a) has at least one scenario-table row	Pass: Non-trivial steps 2 and 3 both have scenario-table rows.	N/A
K5 Every analyzed use case traces to at least one functional requirement	Pass for UC-05: UC-05 traces to FR-01 and FR-10.	N/a
K6 The last messages of each diagram make the use case postcondition true	Fail: The last sequence message presents search results, but the UC-05 postcondition says the gamer selects a game and sees the game page. The current sequence stops too early.	Extend the scenario table/sequence diagram through game selection and opening the game page, or change the analyzed scope/postcondition consistently.

6 High-Level Design (Architecture)

Chapter 8. The rule now reverses: a high-level design that does not say where things run, what they are built from and what that choice cost has not been done.

6.1 Architectural drivers: quality-attribute scenarios

QAS	Quality attribute	Stimulus	Artifact	Response	Response measure	From
QAS-01	Performance	20 users simultaneously searching on normals conditions	Search subsystem	Returns matching game results	At least 95% of searches return within 3 seconds across 100 test searches	QR-01
QAS-02	Usability / Responsive design	A user opens a main page at 360 px or 1366 px viewport width	Web interface	Displays the page correctly	100% of tested main pages have no clipped or overlapping content at both widths	QR-02
QAS-03	Security	An unauthorized or unauthenticated user attempts a moderator-only action	Authorization / moderation subsystem	Denies access	100% of tested unauthorized attempts are blocked	QR-03

Architectural constraints. CON-01 and CON-02 shape the architecture by requiring the system to comply with the access restrictions of external platforms and by preventing unauthorized access to private or restricted third-party data. External game and review data must therefore be accessed only through approved interfaces and controlled components of the system.

6.2 Architectural style and alternatives

Style considered	Drivers it serves	What it costs	Verdict
Layered (3-Tier) with MVC-style separation	Supports QAS-01 performance, QAS-02 usability, and QAS-03 security by separating UI, application logic, authorization, and data access	Requires maintaining clear interfaces and multiple layers	Chosen
Microservices	Could separate search, reviews, moderation, and external data independently	Adds deployment, networking, monitoring, and coordination overhead that is unnecessary for the current scope	Rejected
Event-driven	Could support asynchronous updates and decouple some processing	Adds event infrastructure and complexity that the current use cases do not require	Rejected

The team selected a layered client-server architecture with MVC-style separation. This architecture supports QAS-01 by keeping search and data-access logic centralized on the server, supports QAS-02 by separating the presentation layer from application logic, and supports QAS-03 by keeping authorization decisions on the server rather than relying on the client. Microservices and event-driven architectures were rejected because their added deployment and coordination complexity would not provide enough benefit for the current project scope.

6.3 Component view

6.3 Component View

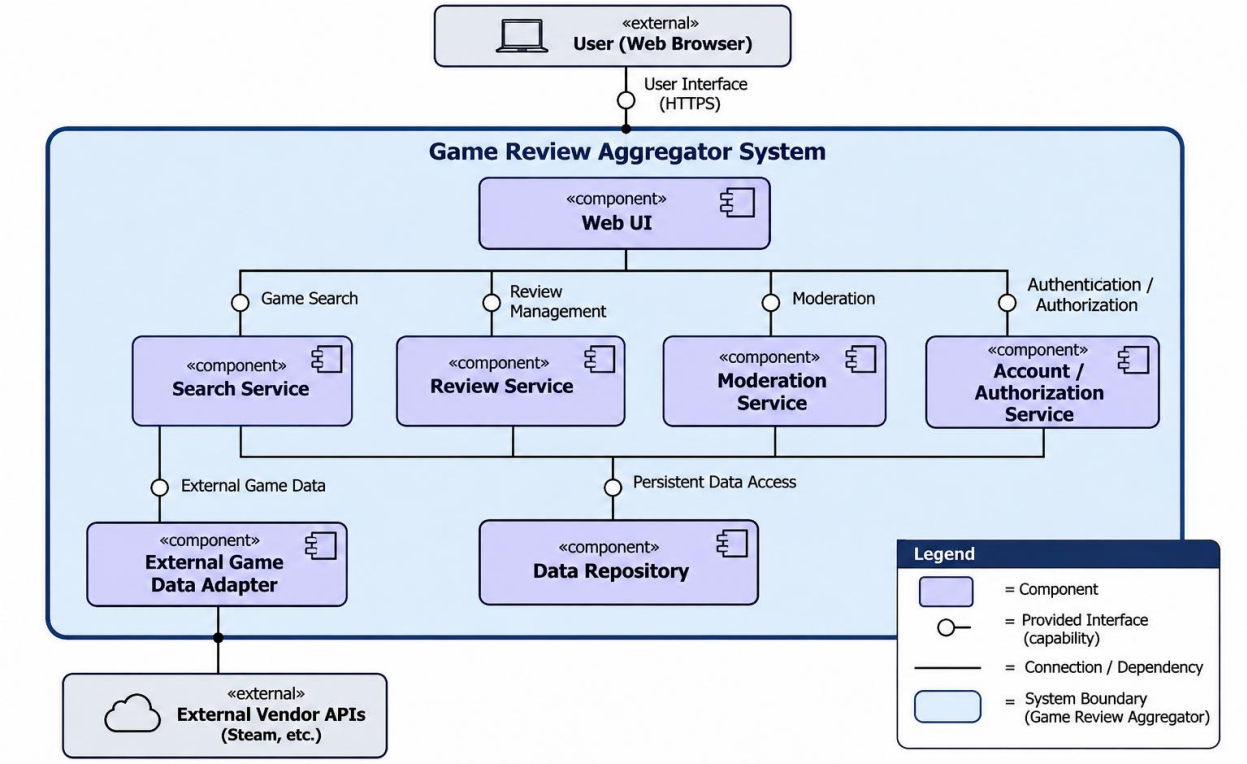


Figure 6.3 Component diagram: components, provided and required interfaces, external systems

Component	Responsibility (one sentence)	Provides	Requires	Answers to (UC / QAS / ADR)
Web UI	Presents search, game details, reviews, account, and moderation views to users	User Interface	Search Service, Review Service, Moderation Service, Account/Auth Service	UC-05, UC-06, UC-02, QAS-02
Search Service	Handles game search, filtering, and matching results	Game Search	Game Data Repository, External Game Data	UC-05, FR-01, FR-10, QAS-01
Review Service	Handles review creation, retrieval, sorting, and aggregate ratings	Review Management	Review Repository, Game Data Repository	UC-02, UC-06, FR-03–FR-07

Component	Responsibility (one sentence)	Provides	Requires	Answers to (UC / QAS / ADR)
Moderation Service	Handles reporting and moderator-only content actions	Moderation	Account/Auth Service, Review Repository	FR-08, FR-09, QAS-03
Account / Authorization Service	Handles user identity and permission checks	Authentication / Authorization	User Repository	QAS-03
Data Repository	Stores users, games, reviews, lists, and related application data	Persistent Data Access	Database	UC-02, UC-05, UC-06, QR-04
External Game Data Adapter	Retrieves permitted game/review information from third-party vendors	External Game Data	Vendor APIs	FR-01, FR-02, CON-01, CON-02

6.4 Deployment view

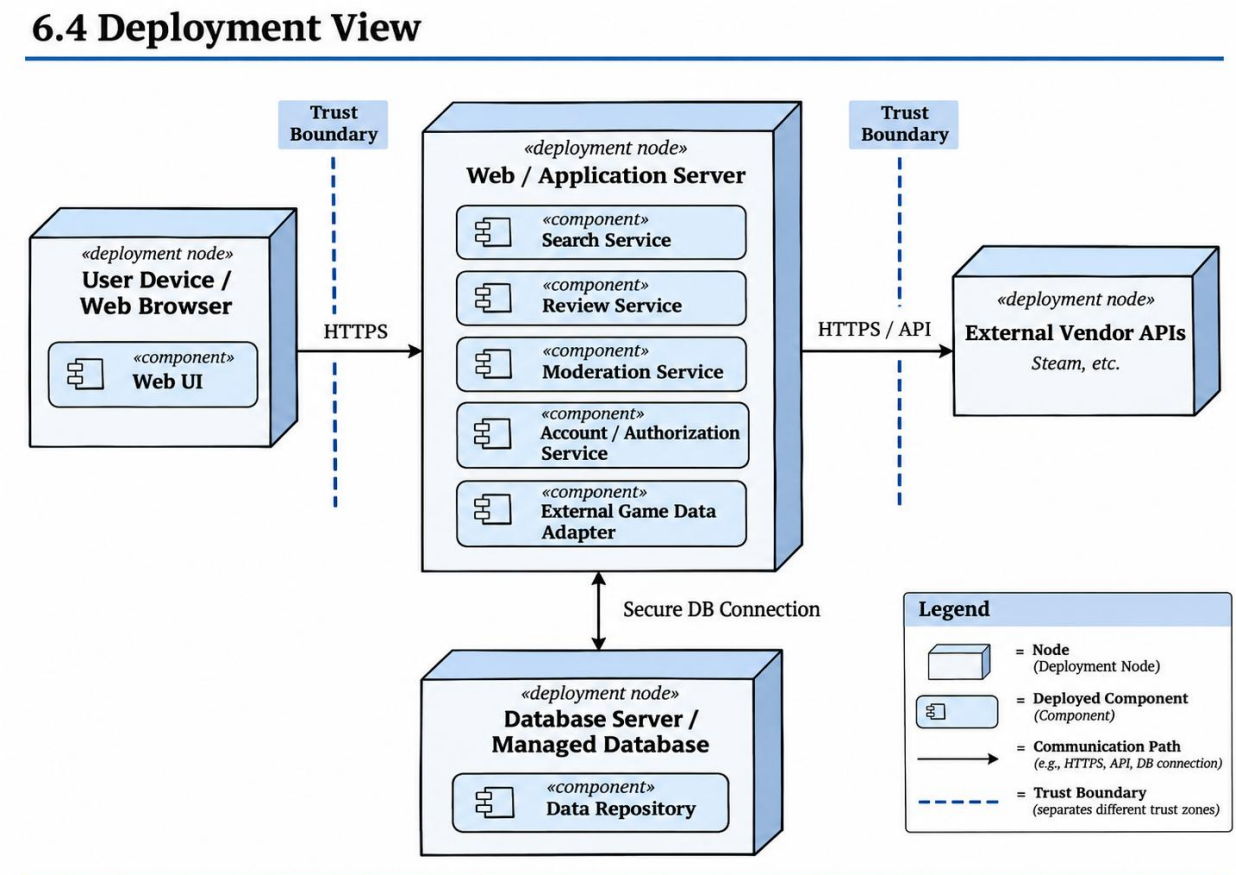


Figure 6.4. *Deployment diagram: nodes, deployed components, communication paths, protocols, and trust boundaries.*

Figure 6.4 UML deployment diagram: nodes, execution environments, deployed artifacts, communication paths with protocols, trust boundaries

Node	What runs on it (artifacts, components)	Communicates with (protocol)	Trust zone
User Device / Web Browser	Web UI	Web/Application Server via HTTPS	User-controlled / untrusted
Web / Application Server	Search Service, Review Service, Moderation Service, Account/Auth Service, External Game Data Adapter	Browser via HTTPS; Database via database connection; Vendor APIs via HTTPS	Team-controlled application zone
Database Server / Managed Database	Data Repository	Web/Application Server via database protocol/secure connection	Team-controlled data zone
External Vendor APIs	Steam and other permitted external game/review data sources	External Game Data Adapter via HTTPS/API	External / third-party trust zone

6.5 From analysis to architecture

Analysis object (from 5.1 and 5.2)	Component that owns it (6.3)	Note (split, merged, external)
Gamer	Account / Authorization Service	User/account identity is managed here
Game	Search Service	Main domain object used for searching and displaying game information
GameReview	Review Service	Review creation, retrieval, sorting, and ratings belong here
Filter	Search Service	Used to refine game searches
List	Review Service	User-created game lists are treated as user content
SearchController	Search Service	Interaction/control object merged into the Search Service
SearchQuery	Search Service	Search interaction object used to represent search criteria

Analysis object (from 5.1 and 5.2)	Component that owns it (6.3)	Note (split, merged, external)
SearchResult	Search Service	Search interaction object used to represent matching results

6.6 Design principles applied

Principle	Where it shows in your design	Evidence (component, interface, ADR)
Separation of concerns	Search, reviews, moderation, account/authorization, UI, and external-data access are handled by separate components instead of one large component	Component view 6.3
Information hiding	Vendor-specific API details are hidden behind the External Game Data Adapter, so other components do not need to know how each vendor API works	External Game Data Adapter
Low coupling	Web UI communicates with application services rather than directly accessing the database or vendor APIs	Web UI interfaces / Component view 6.3
High cohesion	Search Service contains search and filtering responsibilities, while Review Service contains review and rating responsibilities	Search Service and Review Service
Design for change	External vendor access is isolated in the External Game Data Adapter, so a provider can be changed or added without redesigning the rest of the system	External Game Data Adapter
Keep it simple (KISS)	The design uses a layered client-server structure rather than microservices or event-driven architecture because the current project scope does not require the added complexity	Section 6.2 / ADR-01

7 Design Rationale

7.1 Architecture decision records

At least three. ADR-01 records the style decision from 6.2. Use the format you practiced in the Lesson 16 workshop, written about your own project. Copy the block for each decision.

ADR-01 Layered (3-Tier)

Status	Accepted
Date and owner	11/8/2026, Ethan Nguyen
Context (drivers)	QAS-3 since the system needs good performance, usability, and security.
Options considered	Different server architectures including layered architecture, microservices, event-driven architecture.
Decision	Implemented the layered approach
Consequences	This decision favors security and usability by requiring maintaining clear interfaces and multiple layers, but this leads to maintenance between layers
Evidence	Spike 1, uses a 3-tier architecture (client, server, database)
Related	ADR-02, QAS-01, QAS-02, QAS-03

ADR-02 External-data Manipulation

CON-01, CON-02 Status	Accepted
Date and owner	11/8/2026, Rohith Rajan
Context (drivers)	QAS 1, System is based on game metadata and it also reviews information from the external vendors like Steam.
Options considered	Each app calls its own vendor directly or centralized integration layer
Decision	Handle external calls through a separate integration layer
Consequences	It favors maintainability by having a separate integration layer for vendor communication which leads to the extra cost of a layer which could increase search latency
Evidence	You can see evidence for this through documentation from Steamworks Web API [R1] and Steam User Reviews API [R2] which support the need for external API integration.
Related	QAS-01, CON-01, CON-02

ADR-03 User Authorization

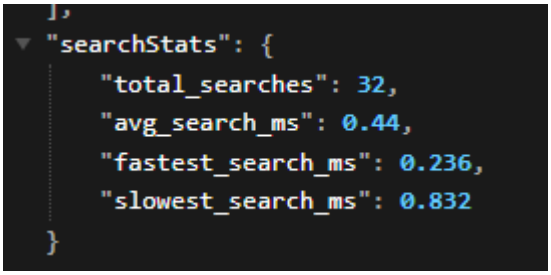
Status	Accepted

Date and owner	11/8/2026, Rohith Rajan
Context (drivers)	QAS 3 requires that the system will block all unauthorized mod access actions
Options considered	The options we looked at were frontend-only authorization and server-side authorization.
Decision	Moderator permissions will be managed in the application layer.
Consequences	Security is favored due to authorization checks being done in the server/application layer, but it costs additional permission checking and management overhead
Evidence	There is none yet leaving the question: Can the server layer stop 100% of unauthorized attempts to do moderator actions?
Related	QAS-03, QR-03

7.2 Trade-off summary

Quality favored	Architectural move	Cost accepted	Recorded in
Performance/Usability	3-tier layer with MVC style of separation	Additional interfaces	ADR-01
Maintainability	External vendor communication moved to new integration layer	Possible extra search latency	ADR-02
Security	Enforced moderator permissions with server-side authorization	Additional permission checks and role management	ADR-03

8 Prototype Evidence: Architecture Spike

Spike ID and title	S-01, architecture spike
Risk or decision it addresses	QAS-01, ADR-01
Question (answerable yes or no)	Will a three-tier system work for our system? Can we get data from external vendors such as Steam, Itch, and Roblox? Can we search by game titles, and is it faster than 3 seconds?
Set-up	/spikes/S-01/src. Used node.js and Sqlite.
How it was measured	Measured the execution times of queries on the sqlite database from API get requests
Result (numbers)	 Average search time was 0.44 milliseconds, which is less than the 3 second quality requirement in QAS-01. Full results in /spikes/S-01/metrics.json
What it changed or confirmed	Confirms assumption A1 from Section 2.1. Confirms that QAS-01 is satisfied. Confirms that a three-tier architecture works for our system.
Where to see it	PR #54, Commit hashes: 10f6f0e, /spikes/S-01/

9 Traceability

9.1 Extended traceability matrix

Requirement	Use case	Analysis classes	Component(s)	QAS / ADR	Evidence
FR-01	UC-05	SearchController, SearchQuery, Game, SearchResult	Search Service, External Game Data Adapter	QAS-01, ADR-01	S-01 spike
FR-02	UC-06, UC-07	Game, SearchResult	Search Service, Web U	ADR-01	Section 6.3 / 6.4
FR-03	UC-06	GameReview, Game	Review Service, Web UI	ADR-01	Section 6.3
FR-04	UC-02	Gamer, GameReview, Game	Review Service, Web UI	ADR-01	Section 6.3
FR-05	UC-04	GameReview	Review Service	ADR-01	Section 6.3
FR-06	UC-06, UC-07	GameReview, Game	Review Service	ADR-01	Section 6.3
FR-07	UC-06	GameReview	Review Service	ADR-01	Section 6.3
FR-08	UC-19, UC-20	GameReview, Gamer	Moderation Service, Account/Auth Service	QAS-03, ADR-03	Section 6.3
FR-09	UC-16, UC-17, UC-18	GameReview, Gamer	Moderation Service	QAS-03, ADR-03	Section 6.3
FR-10	UC-05	Filter, SearchQuery, Game	Search Service	QAS-01, ADR-01	S-01 spike
QR-01	UC-05	SearchController, SearchQuery, SearchResult	Search Service	QAS-01, ADR-01	S-01 spike
QR-02	UC-05 / UC-06 / UC-02	N/A	Web UI	QAS-02, ADR-01	Section 6.3 / 6.4

Requirement	Use case	Analysis classes	Component(s)	QAS / ADR	Evidence
QR-03	Moderator-related use cases	Gamer / authorization interaction	Moderation Service, Account/Auth Service	QAS-03, ADR-03	Section 6.3
QR-04	UC-02 / UC-06	GameReview	Review Service, Data Repository	ADR-01	Section 6.3 / 6.4
CON-01	UC-05 / UC-06	Game, GameReview	External Game Data Adapter	ADR-02	Steam docs / Section 6.3
CON-02	UC-05 / UC-06	Game, GameReview	External Game Data Adapter	ADR-02	Steam docs / Section 6.3

9.2 Gap analysis

Gap	Items	What you did about it
Requirements with no component	None identified	All listed requirements were mapped to at least one architecture component in Section 9.1
Components with no requirement, QAS or ADR	None identified	Each Section 6 component traces to at least one functional requirement, QAS, constraint, or ADR
Use cases not analyzed (and why that is acceptable)	Moderator and social/list-related use cases not selected in Section 4	Only UC-02, UC-05, and UC-06 were selected for detailed analysis because they represent the core product flows and highest-value functionality; remaining use cases still trace to architecture components

10 Project Plan (updated; was P1 section 4)

10.1 Roles and working agreement

Unchanged since v0.1

10.2 Plan to P3

Milestone	Date	Owner	Done when
Detailed design for core components	11/16/2026	Team / assigned component owners	Design class diagrams and detailed sequence diagrams are completed for the Search, Review, Moderation, and Account/Authorization components
Implement core backend and data access	11/23/2026	Backend owners	Search, review submission/retrieval, moderation actions, authorization checks, and database access are working in the repository
Implement web interface and connect to services	11/30/2026	Frontend owners	Users can search for games, view game/review information, submit reviews, and access moderation features through the web interface
Integrate external game-data source	12/3/2026	Integration owner	At least one permitted vendor API is connected through the External Game Data Adapter and returns usable game/review data
Verify P3 functionality and prepare submission	12/7/2026	Team	Core MUST requirements work end-to-end, diagrams/documentation are updated, repository evidence is complete, and the P3 milestone is ready for submission
Final P3 submission	12/9/2026	Team / submitter	Final document, code, diagrams, and repository are ready and submitted

10.3 Risks (updated)

Risk	Status since P1	Likelihood / impact	Mitigation and owner
Review Bombing & Spam	Unresolved	Low	Implement rate limiting, captcha for review submissions, and automated keyword flagging to quarantine suspicious reviews before publication.
Traffic Spikes on Release Days	Unresolved	Low	Design the architecture for horizontal scaling and cache static game information/average ratings to reduce database load during peak times.
Moderator Bottlenecks	Resolved	Low	Build efficient bulk-moderation tools, prioritize flagged content, and implement a tiered trust system where veteran reviewers bypass manual approval.
Reviewer Abandonment	Resolved	Very Low	Conduct usability testing on the submission form early in development to ensure rating options and text fields are straightforward for non-technical writers.
Moderation Scope Creep	In progress	Medium	Strictly limit the initial moderation tools to basic approve, reject, and flag functions, explicitly defining what is excluded from the current scope.
External Vendor Server Crash	Unresolved	High	Temporarily halt new reviews for games in the affected server

10.4 Repository

AgentRachel/game-review-aggregator: Software Engineering project for aggregating and presenting trustworthy video game review information.

10.5 Use of AI tools in P2

Member	Tool	What it was used for	What you checked or changed
Terrell Johnson	Gemini Argon 4	Assisted in building spike S-01. Outlined basic code structure for setting up a test database, calling vendor APIs, and creating the endpoint for searching. Generated a simple, basic html to test searches on local host.	Went over and verified all generated output. Added metric tracking to measure database aggregation and game search latency.
Umar Shaikh	GPT 5.6	Had it generated visual diagrams	Checked if diagram was accurate

11 Open Questions (updated; was P1 section 5)

ID	Question	Raised by	Owner and due date	Status or answer	Assumption meanwhile
Q	How would a new gaming vendor enter the system	Ethan Nguyen	Ethan Nguyen, 11/8/2026	They would apply to the system and after review they would be admitted or denied	They would just enter after signing into the system
Q	How would a vendor leave the system	Ethan Nguyen	Ethan Nguyen, 11/8/2026	They would apply to the system and after acknowledging the request they can leave the system	They would simply leave the system whenever they wanted to

12 Baseline v1.0 Declaration (was P1 section 6)

We declare this document, version 1.0, the baseline for P2. Sections 1 to 9 describe what we will build in P3. Any later change to them is recorded in section 0.2 with its reason.

Team member	Signature or initials	Date
Terrell Johnson	TKJ	11/8/2026
Chayeol Jung	CYJ	11/8/2026
Umar Shaikh	UAS	11/8/2026
Mary Demas	MAD	11/8/2026
Rohith Rajan	RR	11/8/2026
Ethan Nguyen	EKN	11/8/2026